

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 13

CI/CD Pipelines with GitHub Actions

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Microservices Course — Spring Boot & Spring Cloud Professional Edition

Session 13 — CI/CD Pipelines with GitHub Actions

From Code → Test → Docker Image → Container Registry

1. Session Overview

In previous sessions, we built and containerized our microservices.

We now have applications such as:

- `product-service`
- `order-service`
- `config-server`
- `eureka-server`
- `api-gateway`

We also learned how to create Docker images for our services.

But there is an important problem.

How do we deliver new versions of our services?

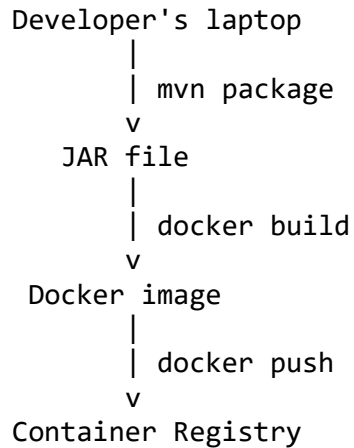
Imagine that a developer changes `product-service`.

Normally, the developer may have to:

1. Pull the latest code.
2. Run the tests.
3. Build the application.
4. Build the Docker image.
5. Choose an image tag.
6. Log in to Docker Hub or another registry.
7. Push the image.
8. Tell another person that a new image is available.
9. Eventually deploy the image.

If developers do these steps manually every time, mistakes are very easy to make.

For example:



What if the developer forgets to run the tests?

What if the developer uses Java 17 locally but production uses Java 21?

What if the Docker image is tagged incorrectly?

What if the developer forgets to push the image?

What if two developers follow different procedures?

We need automation.

That is where **CI/CD** comes in.

2. Learning Objectives

By the end of this session, you should be able to:

CI/CD Fundamentals

- Explain what CI and CD mean.
- Explain why manual deployment is risky.
- Understand the basic CI/CD pipeline.
- Understand the difference between CI and deployment.

GitHub Actions

- Explain what GitHub Actions is.
- Understand workflows, jobs, steps, and runners.
- Create a GitHub Actions workflow.
- Configure a workflow to run automatically.
- Run Maven tests inside GitHub Actions.

Quality Gates

- Explain what a Quality Gate is.
- Understand why tests should stop the pipeline when they fail.
- Understand needs.
- Understand continue-on-error.
- Distinguish blocking checks from informational checks.

Docker

- Build a Docker image inside GitHub Actions.
- Understand Docker Buildx.
- Understand image tags.
- Push a Docker image to a container registry.

Container Registry

- Understand what a container registry is.
- Understand GitHub Container Registry (GHCR).
- Understand Docker Hub.
- Understand registry authentication.
- Understand why credentials must not be stored directly in YAML.

Architecture

Understand why Session 13 stops at:

```
Code
↓
Test
↓
Build Docker Image
↓
Push Image to Registry
```

and why deployment to Kubernetes belongs to the GitOps process introduced in Session 14.

3. Before We Start: The Big Picture

Before writing any GitHub Actions YAML, we need to understand the complete process.

Imagine a developer changes:

product-service

The developer commits the change:

```
git add .
git commit -m "Add product search"
git push
```

GitHub receives the code.

GitHub Actions can automatically start a pipeline.

The pipeline performs:

```
Git Push
  ↓
GitHub Actions
  ↓
Checkout Code
  ↓
Build
  ↓
Run Tests
  ↓
Quality Gate
  ↓
Build Docker Image
  ↓
Push Docker Image
  ↓
Container Registry
```

For this session, the pipeline stops here.

SESSION 13

```
Developer
  |
  | git push
  v
GitHub Repository
  |
  v
GitHub Actions
  |
  +----> Checkout
  |
  +----> Build
  |
  +----> Test
  |
  +----> Quality Gate
  |
  +----> Docker Build
  |
```

```
+----> Docker Push
|
v
Container Registry
```

In Session 14, another process will take over:

```
Container Registry
|
v
GitOps Repository
|
v
Argo CD
|
v
Kubernetes Cluster
```

This separation is extremely important.

4. What Does CI/CD Mean?

4.1 CI — Continuous Integration

Continuous Integration, or CI, means that developers frequently integrate their code into a shared repository and automatically validate the changes.

For example:

```
Developer A
|
| push
v
GitHub
|
v
Automated Tests
```

Another developer does the same:

```
Developer B
|
| push
v
GitHub
|
v
Automated Tests
```

The important idea is:

Every change is automatically checked.

Instead of waiting two weeks and discovering hundreds of problems at once, we detect problems early.

5. What Does CD Mean?

CD can mean:

Continuous Delivery

The software is automatically prepared and is always ready to be deployed.

Or:

Continuous Deployment

The software is automatically deployed to an environment after passing the required checks.

For this course, we will separate the responsibilities.

Session 13 focuses mainly on:

Continuous Integration

+

Build and publish the deployable artifact

Session 14 introduces:

GitOps-based deployment

Therefore:

Session 13

Code

↓

Test

↓

Build

↓

Docker Image

↓

Registry

Then:

Session 14

Registry
↓
Git Manifest
↓
Argo CD
↓
Kubernetes

This separation is intentional.

6. The Problem with Manual Deployment

Let's start with a real-world scenario.

It is Friday at 4:00 PM.

The team has been working on a new feature for two weeks.

The feature is ready.

Ahmed says:

“I will deploy it.”

He builds the application:

```
mvn package -DskipTests
```

Why did he skip the tests?

Because:

“The code works on my machine.”

He now copies the JAR to the server.

Then:

```
SSH into server  
↓  
Stop old application  
↓  
Copy new JAR  
↓  
Start application
```

The application fails.

The server uses Java 17.

Ahmed developed using Java 21.

Now there is a compatibility problem.

He needs to rollback.

He searches for the previous JAR.

He restarts the old version.

The team loses 45 minutes.

More importantly, everyone becomes nervous about deploying.

7. Why Manual Deployment Is Dangerous

Manual deployment has several problems.

Problem 1 — Human error

People forget steps.

For example:

- Forgot to run tests
- Forgot to build Docker image
- Forgot to push image
- Wrong image tag
- Wrong server
- Wrong configuration

Problem 2 — Environment differences

The developer's machine may have:

- Java 21
- Maven 3.x
- Docker version X

while the server has:

- Java 17
- Maven version Y
- Docker version Z

The application may work locally but fail somewhere else.

Problem 3 — Inconsistent process

Developer A may deploy like this:

```
mvn package
docker build
docker push
```

Developer B may do:

```
mvn package -DskipTests
docker build
```

Developer C may forget the Docker push.

The process depends on people.

Problem 4 — Difficult auditing

Later someone asks:

Which version was deployed?

You may have to search manually.

With CI/CD, we can associate an image with a Git commit.

For example:

```
sha-a81c92d
```

This means:

This Docker image was built from a specific Git commit.

That makes the release traceable.

8. Manual Process vs CI Pipeline

Manual Deployment	CI Pipeline
Developer performs steps	Pipeline performs steps
Developer's machine	Controlled runner
Tests may be skipped	Tests automatically run
Manual Docker build	Automated Docker build
Manual image tagging	Consistent tagging
Manual push	Automated push

Manual Deployment	CI Pipeline
Easy to make mistakes	Repeatable
Difficult to audit	Git + pipeline history
Fear of deployment	Confidence in deployment

The important idea is:

Automation is not only about saving time. It is about making the process repeatable and reliable.

9. What Is a Pipeline?

A **pipeline** is a sequence of automated operations.

For example:

CI PIPELINE

```
Git Push
  |
  v
Checkout
  |
  v
Compile
  |
  v
Test
  |
  v
Quality Gate
  |
  v
Docker Build
  |
  v
Docker Push
```

Each stage has a responsibility.

10. What Is a GitHub Actions Workflow?

GitHub Actions is GitHub's automation platform.

It allows us to define automated workflows inside our repository.

A workflow is usually stored in:

```
.github/  
└─ workflows/  
    └─ product-service-ci.yml
```

The file contains instructions such as:

When should the workflow run?

What machine should run it?

What commands should it execute?

What happens if a test fails?

How should the Docker image be built?

Where should the image be pushed?

11. Important GitHub Actions Terminology

Before writing YAML, learn these four terms.

Workflow

A complete automation process.

Example:

Product Service CI

Job

A group of related steps.

For example:

test

and:

build-and-push

could be two jobs.

Step

One operation inside a job.

For example:

Checkout code

or:

Run Maven tests

Runner

The machine that executes the job.

For example:

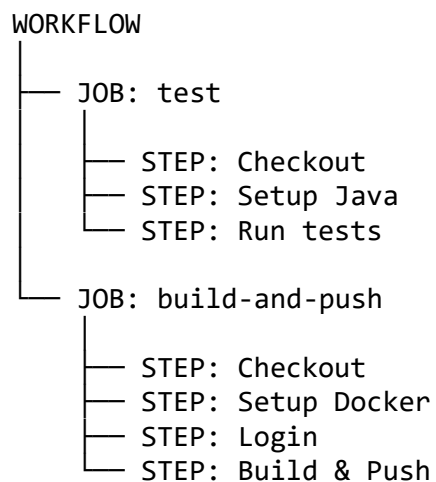
`runs-on: ubuntu-latest`

means:

Run this job on a GitHub-hosted Ubuntu machine.

12. Workflow → Job → Step

Think about it like this:



This hierarchy is one of the most important things to understand in GitHub Actions.

13. Jobs vs Steps

Suppose we have:

```
jobs:
  test:
    steps:
      - checkout
      - setup-java
      - test
```

The three operations are steps in the same job.

They normally execute sequentially:

```
checkout
  ↓
setup-java
  ↓
test
```

Jobs are different.

For example:

```
jobs:
  test:
  security:
  build:
```

Jobs can run independently or can depend on each other.

We can explicitly say:

```
needs: test
```

meaning:

Do not start this job until the test job has successfully completed.

14. What Is a Quality Gate?

A **Quality Gate** is a condition that must be satisfied before the pipeline is allowed to continue.

For our pipeline:

```
Run Tests
  |
  | PASS
```

v
Build Docker Image

But:

Run Tests
|
| FAIL
v
STOP

The test result is therefore a Quality Gate.

15. Why Should Tests Stop the Pipeline?

Imagine this:

Tests FAILED
|
v
Docker image built
|
v
Docker image pushed
|
v
Deployment

We have created a deployable artifact from broken code.

That defeats the purpose of automated testing.

Instead:

Tests FAILED
|
v
STOP PIPELINE

This is called **fail-fast** behavior.

The pipeline tells the developer as early as possible:

Your change is not ready.

16. Quality Gate Example

Our rule is:

`mvn test passes`



`continue`

If:

`mvn test fails`

then:

Docker build does NOT happen.

Docker push does NOT happen.

This gives us:

Code



Test



PASS?

├── NO → STOP

└── YES



Docker Build



Docker Push

17. `continue-on-error`

GitHub Actions supports:

`continue-on-error: true`

This means:

If this step fails, do not necessarily stop the job.

This can be useful for informational checks.

For example:

Checkstyle

SpotBugs

Coverage report

depending on the team's policy.

But we should NOT normally use it for our main unit tests.

Avoid:

```
- name: Run tests
  run: mvn test
  continue-on-error: true
```

because then tests can fail while the pipeline continues.

For our course:

Unit tests = blocking

Therefore:

Test failure → pipeline failure

18. Create the GitHub Actions Workflow

Now we implement it.

Inside the repository root, create:

```
.github/
├─ workflows/
│   └─ product-service-ci.yml
```

Important:

The workflow belongs in the repository's `.github/workflows` directory.

It does NOT belong inside:

`services/product-service/.github/`

19. Our First Workflow

Start with a very small workflow.

```
name: Product Service CI
```

```
on:
  push:
    branches: [main]
```

```
jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test -pl services/product-service -B
```

Do not copy this blindly.

Let's understand every part.

20. name

name: Product Service CI

This is the workflow's display name.

GitHub will show this name in the Actions interface.

It is for humans.

21. on

on:

This defines:

When should the workflow run?

For example:

```
on:
  push:
    branches: [main]
```

means:

Run the workflow when code is pushed to the `main` branch.

22. Pull Requests

We can also run the workflow when someone creates or updates a pull request.

```
on:
  push:
    branches: [main]

  pull_request:
    branches: [main]
```

Now we have:

```
Push to main
  ↓
CI

Pull Request → main
  ↓
CI
```

This is useful because we can detect problems before merging.

23. Other Common Triggers

GitHub Actions supports many triggers.

For example:

Push

```
push:
```

Run when code is pushed.

Pull Request

```
pull_request:
```

Run when a pull request changes.

Manual

```
workflow_dispatch:
```

Allows a user to manually start the workflow.

Scheduled

A workflow can also run on a schedule.

For example, a nightly security check.

For this session, our main focus is:

```
push
pull_request
```

24. Path Filtering

Our repository contains multiple microservices.

For example:

```
services/
├── product-service/
├── order-service/
├── payment-service/
└── customer-service/
```

Suppose we modify only:

```
services/product-service/
```

We do not necessarily want the Product Service CI workflow to run because someone changed:

```
services/order-service/
```

We can use:

```
paths:
- 'services/product-service/**'
```

Example:

```
on:
  push:
    branches: [main]
    paths:
      - 'services/product-service/**'

  pull_request:
    branches: [main]
    paths:
      - 'services/product-service/**'
```

Now the workflow focuses on Product Service changes.

25. Environment Variables

We can define shared values:

```
env:  
  JAVA_VERSION: '21'  
  REGISTRY: ghcr.io  
  IMAGE_NAME: ${github.repository}/product-service
```

Now instead of writing:

21

everywhere, we can use:

```
${env.JAVA_VERSION}
```

Similarly:

```
${env.REGISTRY}
```

means:

Use the value of the REGISTRY environment variable.

26. What Is GHCR?

Our pipeline needs somewhere to store Docker images.

That location is called a **container registry**.

A container registry is a service that stores container images.

Examples include:

- GitHub Container Registry
- Docker Hub
- cloud provider registries

For this course, we use:

GHCR

which means:

GitHub Container Registry

The registry address is:

ghcr.io

So an image could look like:

ghcr.io/my-company/product-service:latest

or:

ghcr.io/my-company/product-service:sha-a81c92d

27. What Is a Docker Image?

A Docker image is the packaged version of our application.

For example:

Spring Boot application

+

JRE/JDK environment

+

dependencies

+

configuration

+

Dockerfile instructions

↓

Docker Image

The image can then be used to create containers.

For example:

Docker Image

↓

Container

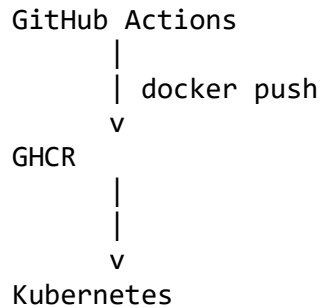
28. Why Store the Image in a Registry?

Suppose GitHub Actions builds:

product-service

The Kubernetes cluster needs access to that image.

We therefore push it to a registry:



The registry acts as a central storage location for container images.

29. Docker Hub vs GHCR

Two possible choices are:

Docker Hub

or:

GitHub Container Registry

Conceptually they serve the same main purpose:

Store Docker/container images

For this course we use:

GHCR

because our source code is already hosted on GitHub.

30. The Test Job

Now let's create the first job.

```
jobs:
  test:
    name: Test & Quality Gate
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK
```

```
    uses: actions/setup-java@v4
    with:
      java-version: '21'
      distribution: 'temurin'
      cache: maven

  - name: Run tests
    run: mvn test -pl services/product-service -B
```

Let's understand it step by step.

31. jobs

jobs:

This section defines the jobs in the workflow.

We currently have:

test:

So the job ID is:

test

32. runs-on

runs-on: ubuntu-latest

This tells GitHub:

Run this job on a GitHub-hosted Ubuntu runner.

A runner is the machine where our commands execute.

For example:

```
GitHub
  |
  v
Ubuntu Runner
  |
  +-- Java
  +-- Maven
  +-- Docker
  +-- Git
```

We don't have to manage this machine ourselves for this basic setup.

33. Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

A GitHub Actions runner is a temporary machine.

It needs our source code.

This action downloads/checks out the repository so that later steps can execute commands against it.

Without checkout, our runner does not automatically have our repository files available.

34. Setup Java

```
- name: Set up JDK
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: maven
```

This prepares Java for our build.

We specify:

```
java-version: '21'
```

because our project uses Java 21.

We use:

```
distribution: 'temurin'
```

which is the Eclipse Temurin JDK distribution.

We also enable:

```
cache: maven
```

to speed up repeated builds by caching Maven dependencies.

35. Run Maven Tests

Now:

```
- name: Run tests  
  run: mvn test -pl services/product-service -B
```

The important command is:

```
mvn test
```

This runs the tests.

The:

```
-pl
```

option means:

Build/test a specific Maven project in a multi-module repository.

In our example:

```
-pl services/product-service
```

means the Product Service module.

The:

```
-B
```

means Maven batch mode.

It is useful in CI because we don't need interactive output.

36. What Happens If a Test Fails?

Suppose we have:

```
10 tests
```

and:

```
9 passed
```

```
1 failed
```

Then:

```
mvn test  
  |  
  v
```

FAILED
|
v
GitHub Actions step fails
|
v
Job fails
|
v
Pipeline stops

The Docker image should not be built.

This is our Quality Gate.

37. First Practical Exercise

Exercise 1 — Create the CI Workflow

Create:

```
.github/workflows/product-service-ci.yml
```

Use the workflow we built above.

Commit it:

```
git add .github/workflows/product-service-ci.yml
```

Then:

```
git commit -m "Add product service CI workflow"
```

Then:

```
git push
```

38. Observe GitHub Actions

Open your GitHub repository.

Go to:

Actions

You should see:

Product Service CI

Click the workflow.

You should see:

Test & Quality Gate

Inside the job:

Checkout code

↓

Set up JDK

↓

Run tests

This is your first automated CI pipeline.

39. Exercise 2 — Make a Test Fail

Temporarily introduce a failing test.

For example:

```
@Test
void shouldFail() {
    assertEquals(1, 2);
}
```

Commit and push.

GitHub Actions should report:

FAILED

The pipeline should stop.

This is important.

You have just demonstrated:

Failed Quality Gate

↓

Pipeline stops

Fix the test and push again.

The workflow should become:

PASSED

40. Now Add Docker

Our pipeline currently does:

```
Code
↓
Test
```

We now want:

```
Code
↓
Test
↓
Docker Build
↓
Docker Push
```

But we need one important rule:

Docker must be built only after tests pass.

41. Multiple Jobs

We can create a second job:

```
jobs:

  test:
    ...

  build-and-push:
    ...
```

Now we have:

test

and:

build-and-push

But there is an important question:

How does GitHub know that:

build-and-push

depends on:

test

We use:

`needs: test`

42. `needs`

Example:

```
build-and-push:  
  needs: test
```

This means:

```
test  
  |  
  | success  
  v  
build-and-push
```

If:

test FAILS

then:

build-and-push

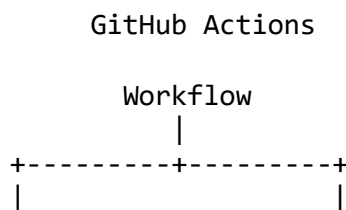
does not run.

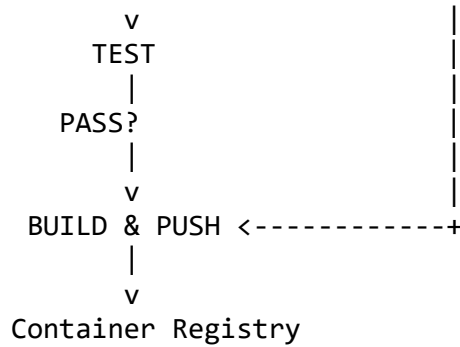
Therefore:

TEST = QUALITY GATE

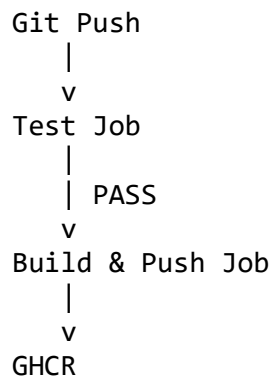
43. Complete Pipeline Structure

Now we have:





More precisely:



44. Docker Buildx

We now need to build the Docker image.

GitHub Actions commonly uses Docker Buildx.

Add:

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
```

What is Buildx?

Buildx is Docker's extended build interface.

It provides capabilities such as:

- advanced build features
- build caching
- multi-platform builds

For our course, the most important point is:

Buildx provides a modern Docker image building mechanism suitable for CI.

45. Login to the Registry

Before pushing the image, GitHub Actions must authenticate with the registry.

For GHCR:

```
- name: Login to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }
```

Let's understand this.

46. registry

```
registry: ${ env.REGISTRY }
```

Our environment variable says:

```
REGISTRY: ghcr.io
```

Therefore:

```
registry = ghcr.io
```

47. github.actor

```
username: ${ github.actor }
```

github.actor represents the GitHub user or identity that triggered the workflow.

48. GITHUB_TOKEN

```
password: ${ secrets.GITHUB_TOKEN }
```

GitHub automatically provides a special token for workflows.

For GHCR, the workflow can use this token when the workflow has the required package permissions.

We therefore do not need to manually create a Docker password just for this GHCR example.

49. Workflow Permissions

The workflow needs permission to publish packages.

Therefore our build job includes:

```
permissions:  
  contents: read  
  packages: write
```

Meaning:

contents: read

The workflow can read repository contents.

And:

packages: write

allows the workflow to publish packages/images to GitHub Packages / GHCR.

50. Docker Hub Authentication

If you choose Docker Hub instead, the authentication model is different.

You would normally store credentials as GitHub Secrets, such as:

```
DOCKER_USERNAME  
DOCKER_TOKEN
```

Then:

```
username: ${ secrets.DOCKER_USERNAME }  
password: ${ secrets.DOCKER_TOKEN }
```

The important security rule is:

Never put passwords, tokens, or access keys directly inside the workflow YAML.

Bad:

```
password: my-real-password
```

Good:

```
password: ${ secrets.DOCKER_TOKEN }
```

51. What Are GitHub Secrets?

GitHub Secrets are protected values stored by GitHub.

Examples:

```
DOCKER_USERNAME  
DOCKER_TOKEN  
AWS_ACCESS_KEY  
API_KEY
```

Instead of putting:

```
my-secret-password
```

in the source code, we reference:

```
${ secrets.MY_SECRET }
```

This keeps sensitive values out of the repository.

52. Image Tags

A Docker image needs a tag.

For example:

```
product-service:latest
```

But latest alone is not enough for reliable release tracking.

Imagine:

```
latest
```

today means:

```
version A
```

and tomorrow:

```
version B
```

Now it is difficult to know which exact image was deployed.

A better approach is to use a Git commit SHA.

For example:

```
product-service:sha-a81c92d
```

This gives us traceability.

We can answer:

Which source-code commit created this image?

53. Image Naming

Suppose:

```
REGISTRY = ghcr.io
```

and:

```
IMAGE_NAME = my-company/my-repository/product-service
```

The complete image can be:

```
ghcr.io/my-company/my-repository/product-service:sha-a81c92d
```

The structure is:

```
registry
|
+---- ghcr.io
      |
      +---- repository/image
            |
            +---- tag
```

54. Docker Metadata Action

Instead of manually creating all tags, we can use:

```
- name: Extract Docker metadata
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
    tags: |
      type=ref,event=branch
```

```
type=sha,prefix=sha-  
type=raw,value=latest,enable=${{ github.ref == 'refs/heads/main' }}
```

This action generates Docker image metadata.

It can create tags based on:

- branch
- commit SHA
- other GitHub context

For example:

```
main  
sha-a81c92d  
latest
```

55. Why Use SHA Tags?

Suppose these commits exist:

```
a81c92d  
b42e711  
c91f220
```

The registry may contain:

```
product-service:sha-a81c92d  
product-service:sha-b42e711  
product-service:sha-c91f220
```

Now we know exactly which image corresponds to which commit.

This becomes extremely useful for:

- debugging
 - rollback
 - auditing
 - deployment tracking
-

56. Build and Push the Docker Image

Now we can use:

```
- name: Build and push Docker image  
  uses: docker/build-push-action@v5
```

```
with:
  context: services/product-service
  push: true
  tags: ${{ steps.meta.outputs.tags }}
  labels: ${{ steps.meta.outputs.labels }}
  cache-from: type=gha
  cache-to: type=gha,mode=max
```

Let's understand each property.

57. context

```
context: services/product-service
```

This tells Docker where the build context is.

Our Dockerfile is associated with:

services/product-service

For example:

```
services/
├─ product-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
```

58. push

```
push: true
```

This tells the action:

After successfully building the image, push it to the configured registry.

Without:

```
push: true
```

we may only build the image locally on the runner.

The image would disappear when the temporary runner is removed.

59. tags

```
tags: ${ steps.meta.outputs.tags }
```

This uses the tags generated by:

docker/metadata-action

For example:

sha-a81c92d

60. Build Cache

We can use GitHub Actions cache:

```
cache-from: type=gha  
cache-to: type=gha,mode=max
```

This can make repeated Docker builds faster.

Conceptually:

First build

↓

Build everything

↓

Cache layers

Second build

↓

Reuse unchanged layers

↓

Build only what changed

61. Complete GitHub Actions Workflow

Now let's put everything together.

```
name: Product Service CI  
  
on:  
  push:  
    branches: [main]  
    paths:  
      - 'services/product-service/**'
```

```
pull_request:
  branches: [main]
  paths:
    - 'services/product-service/**'
```

```
env:
  JAVA_VERSION: '21'
  REGISTRY: ghcr.io
  IMAGE_NAME: ${GITHUB_REPOSITORY}/product-service
```

```
jobs:
```

```
test:
  name: Test & Quality Gate
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up JDK ${JAVA_VERSION}
      uses: actions/setup-java@v4
      with:
        java-version: ${JAVA_VERSION}
        distribution: 'temurin'
        cache: maven

    - name: Run tests
      run: mvn test -pl services/product-service -B
```

```
build-and-push:
  name: Build & Push Docker Image
  runs-on: ubuntu-latest
```

```
needs: test
```

```
permissions:
  contents: read
  packages: write
```

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up Docker Buildx
```

```

    uses: docker/setup-buildx-action@v3

- name: Login to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Extract Docker metadata
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
    tags: |
      type=ref,event=branch
      type=sha,prefix=sha-
      type=raw,value=latest,enable=${ github.ref == 'refs/heads/main' }
}}

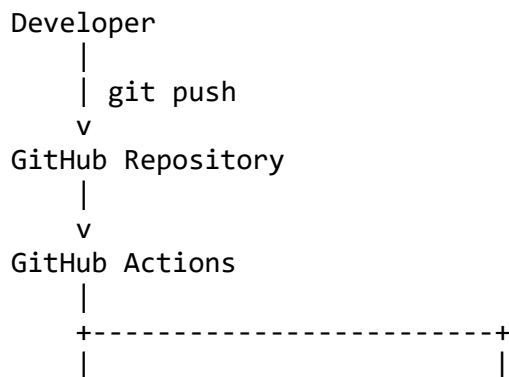
- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: services/product-service
    push: true
    tags: ${ steps.meta.outputs.tags }
    labels: ${ steps.meta.outputs.labels }
    cache-from: type=gha
    cache-to: type=gha,mode=max

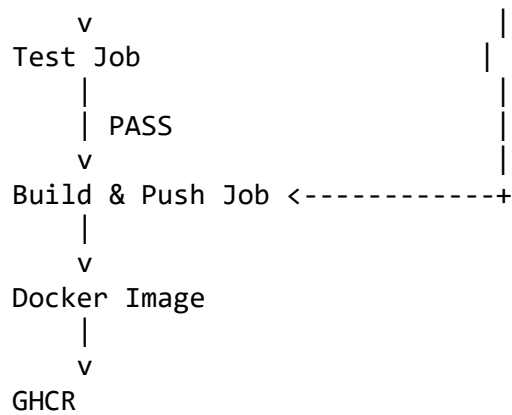
```

62. Understand the Complete Pipeline

Do not memorize the YAML.

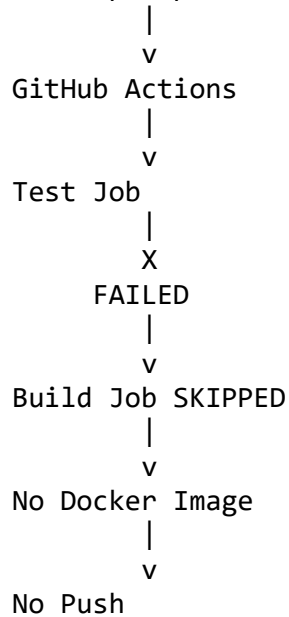
Understand the flow.





63. What Happens When Tests Fail?

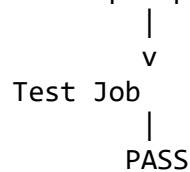
Developer pushes code

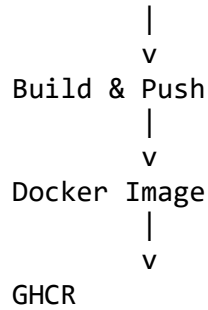


This is exactly what we want.

64. What Happens When Tests Pass?

Developer pushes code





Now the registry contains the new artifact.

65. What Is an Artifact?

An **artifact** is a produced output of a build process.

In our pipeline, an important artifact is:

Docker Image

For example:

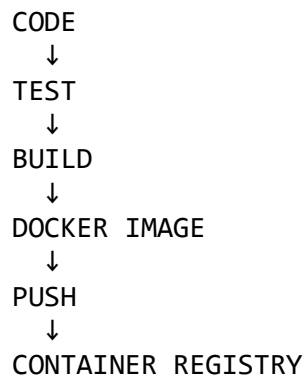
`ghcr.io/company/product-service:sha-a81c92d`

The pipeline's responsibility is to produce this artifact reliably.

66. Where Does Session 13 Stop?

This boundary is extremely important.

Session 13:



Session 14:

CONTAINER REGISTRY
↓
GIT MANIFEST
↓
ARGO CD
↓
KUBERNETES

Therefore, **Session 13 does not deploy directly to Kubernetes.**

67. Why Don't We Use `kubectl apply` Here?

A beginner might think:

After `docker push`:

```
kubectl apply -f k8s/
```

This is technically possible in a push-based CI/CD design.

However, our course deliberately uses GitOps.

The architecture becomes:

```
CI  
|  
+--> build image  
|  
+--> push image  
|  
X--> does NOT directly control Kubernetes
```

Instead:

```
GitOps  
|  
+--> watches Git  
|  
+--> detects desired state  
|  
+--> deploys to Kubernetes
```

This separation will be the main subject of Session 14.

68. CI vs GitOps

Think about the responsibilities this way:

CI	GitOps
Build source code	Manage desired deployment state
Run tests	Synchronize cluster
Build Docker image	Detect drift
Push image	Apply manifests
Produce artifact	Deploy artifact
GitHub Actions	Argo CD

So:

GitHub Actions

=

"Can we build a valid artifact?"

while:

Argo CD

=

"Does the Kubernetes cluster match what Git says?"

69. Important Engineering Decision

Should CI deploy directly?

There are two common approaches.

Option A — Push-based deployment

```
CI
|
+--> Build
|
+--> Test
|
+--> Docker Push
|
+--> kubectl apply
|
v
Kubernetes
```

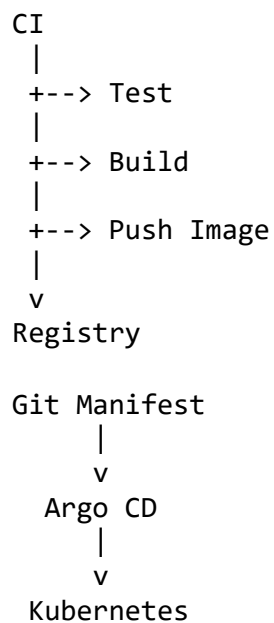
Advantages:

- Simple to understand.
- Easy for small systems.
- Fewer tools.

Disadvantages:

- CI needs Kubernetes credentials.
- Deployment logic is coupled to CI.
- Drift detection is not naturally provided.
- Git may not represent the actual cluster state.

Option B — GitOps



Advantages:

- Git becomes the desired-state source.
- Strong audit trail.
- Drift detection.
- Deployment responsibility is separated from CI.
- CI does not need direct cluster credentials.

Disadvantages:

- More concepts.
- Additional tooling.
- Requires understanding GitOps and Argo CD.

For our course:

We choose GitOps because Kubernetes is our deployment platform and we want a professional deployment architecture.

70. Matrix Strategy — Testing Multiple Java Versions

GitHub Actions can also run the same job against multiple versions.

For example:

```
strategy:
  matrix:
    java: ['17', '21']
```

Then:

```
- name: Set up Java
  uses: actions/setup-java@v4
  with:
    java-version: ${ matrix.java }
    distribution: temurin
```

This can produce:

```
Java 17
  |
  v
Tests
```

```
Java 21
  |
  v
Tests
```

This is called a **matrix strategy**.

71. Why Use a Matrix?

Suppose our library officially supports:

```
Java 17
Java 21
```

We can test both automatically.

Instead of creating two separate jobs manually:

```
test-java17  
test-java21
```

we define:

```
matrix:  
  java: ['17', '21']
```

GitHub creates the combinations for us.

72. Should We Always Use a Matrix?

No.

A matrix increases CI cost and execution time.

If our production application is standardized on:

Java 21

then testing only Java 21 may be sufficient.

Use a matrix when:

We intentionally support multiple versions.

Do not use it just because the feature exists.

This is an engineering decision.

73. Fail-Fast and Matrix Builds

Suppose:

```
Java 17 → FAIL  
Java 21 → PASS
```

Depending on matrix configuration, we can decide whether another matrix execution should continue.

For example:

```
strategy:  
  fail-fast: true  
matrix:  
  java: ['17', '21']
```

With fail-fast enabled, GitHub can cancel in-progress matrix jobs when one fails.

For our production service, if Java 21 is the only supported runtime, a matrix may not be necessary.

74. Quality Checks: Blocking vs Informational

Not every check needs to block deployment.

Think about:

Blocking

Unit tests

Compilation

Security vulnerability above policy threshold

If they fail:

STOP

Informational

Depending on team policy:

Coverage report

Style report

Static-analysis report

These might be reported without blocking.

The important question is:

Is this check a release requirement or an informational report?

75. Common Beginner Mistakes

Mistake 1 — Putting workflow in the wrong location

Wrong:

`services/product-service/.github/workflows/`

Correct:

`.github/workflows/`

at the repository root.

Mistake 2 — Forgetting `needs`

If you write:

```
jobs:
  test:
    ...

  build:
    ...
```

the jobs may run independently.

If build must wait for tests:

```
build:
  needs: test
```

Mistake 3 — Allowing tests to fail

Avoid:

```
continue-on-error: true
```

on the main test step.

Otherwise:

Tests fail

↓

Pipeline continues

↓

Docker image created

That is not our desired Quality Gate.

Mistake 4 — Hardcoding secrets

Never:

```
password: MyPassword123
```

Use:

```
password: ${ secrets.DOCKER_TOKEN }
```

Mistake 5 — Using only `latest`

This:

```
product-service:latest
```

does not tell us which Git commit produced the image.

Prefer traceable tags such as:

```
product-service:sha-a81c92d
```

We may still publish:

```
latest
```

for convenience, but the immutable/traceable SHA tag is important.

Mistake 6 — Building before testing

Bad pipeline:

```
Code
↓
Docker Build
↓
Test
```

Better:

```
Code
↓
Test
↓
Docker Build
```

Why?

Because there is no reason to build and publish an image for code that failed its Quality Gate.

76. Practical Lab

Lab 13 — Automate Product Service CI

Goal

Create a complete CI pipeline for:

product-service

The pipeline must:

1. Run on changes to Product Service.
 2. Checkout the repository.
 3. Install Java 21.
 4. Run Maven tests.
 5. Stop when tests fail.
 6. Build the Docker image.
 7. Tag the image using the Git SHA.
 8. Authenticate with GHCR.
 9. Push the image to GHCR.
-

77. Lab Task 1 — Create the Workflow

Create:

`.github/workflows/product-service-ci.yml`

Start with:

```
name: Product Service CI
```

Add the trigger:

```
on:
  push:
    branches: [main]
```

Then add:

```
jobs:
  test:
```

78. Lab Task 2 — Configure Java

Configure:

Java 21
Temurin
Maven cache

Expected structure:

```
- name: Set up JDK
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: temurin
    cache: maven
```

79. Lab Task 3 — Run the Tests

Add:

```
- name: Run tests
  run: mvn test -pl services/product-service -B
```

Push your workflow.

Check GitHub Actions.

Expected:

Checkout	✓
Setup Java	✓
Run tests	✓

80. Lab Task 4 — Test the Quality Gate

Intentionally break a test.

Push again.

Expected:

```
Test
↓
FAIL
↓
Pipeline stops
```

The Docker build should not run.

Then fix the test.

Push again.

Expected:

Test
↓
PASS

81. Lab Task 5 — Add Docker Build

Add a second job:

`build-and-push:`

Make it depend on:

`needs: test`

Add:

`runs-on: ubuntu-latest`

Then configure Docker Buildx.

82. Lab Task 6 — Configure GHCR

Add:

```
permissions:
  contents: read
  packages: write
```

Then login:

```
- name: Login to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }
```

83. Lab Task 7 — Build and Push

Configure:

`docker/metadata-action`

and:

`docker/build-push-action`

The image should be pushed to:

`ghcr.io`

using a SHA-based tag.

84. Verify the Image

After a successful pipeline:

Go to the GitHub repository.

Open the package/container section.

You should find an image similar to:

`product-service`

with a tag similar to:

`sha-a81c92d`

The exact SHA will be different for your commit.

85. Verify Traceability

Find the commit that triggered the pipeline.

Suppose:

Commit:
`a81c92d`

The registry should contain:

`product-service:sha-a81c92d`

Therefore:

```
Git Commit
  |
  | built by CI
  v
Docker Image
```

This is release traceability.

86. Lab Challenge

Now modify:

product-service

and push a second change.

You should see another image tag.

For example:

sha-a81c92d

sha-b72f410

Now you have two identifiable versions.

This becomes useful later when we discuss:

Deployment

Rollback

GitOps

87. Troubleshooting Guide

Problem: Workflow does not start

Check:

`.github/workflows/`

and verify the YAML file is committed and pushed.

Also check the trigger:

```
on:
  push:
    branches: [main]
```

Are you actually pushing to main?

Problem: Maven cannot find the project

Check:

```
mvn test -pl services/product-service
```

from the repository root.

The path must match the actual project structure.

Problem: Java version mismatch

Check:

```
java-version: '21'
```

and compare it with the Java version used by your project.

Problem: GHCR permission denied

Check:

```
permissions:
  contents: read
  packages: write
```

Also verify that the workflow is authenticating using:

```
secrets.GITHUB_TOKEN
```

Problem: Dockerfile not found

Check:

```
context: services/product-service
```

and verify that the Dockerfile exists in the expected location.

For example:

```
services/product-service/
└─ Dockerfile
```

88. Important Concepts to Remember

At this point, you should be able to explain:

GitHub Actions

GitHub's automation platform for executing workflows.

Workflow

A complete automation process defined in YAML.

Job

A group of steps executed together on a runner.

Step

An individual operation inside a job.

Runner

The machine that executes a GitHub Actions job.

Quality Gate

A condition that must pass before the pipeline continues.

Container Registry

A service that stores and distributes container images.

GHCR

GitHub Container Registry.

Docker Image

A packaged application artifact used to create containers.

SHA Tag

A tag derived from a Git commit, providing traceability.

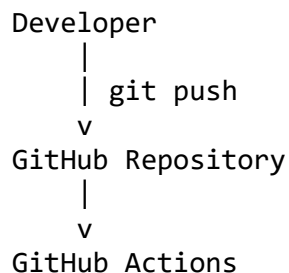
needs

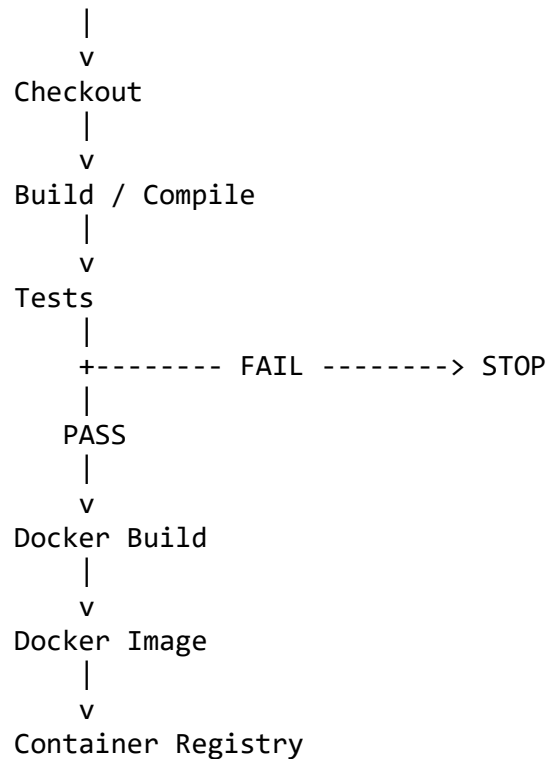
Defines a dependency between GitHub Actions jobs.

89. The Most Important Mental Model

Remember this:

CI PIPELINE





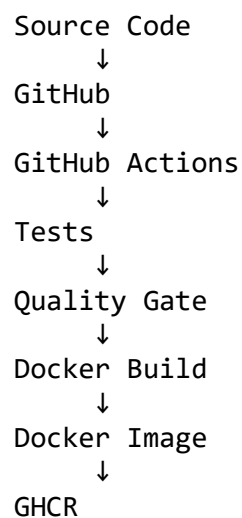
The pipeline is a **safety system**.

It does not merely automate commands.

It protects the system from publishing code that has not passed the required checks.

90. Where We Go Next

At the end of Session 13, we have:

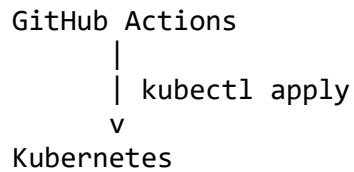


But the image is still sitting in the registry.

We now have another question:

How does this image reach Kubernetes?

One possibility is:

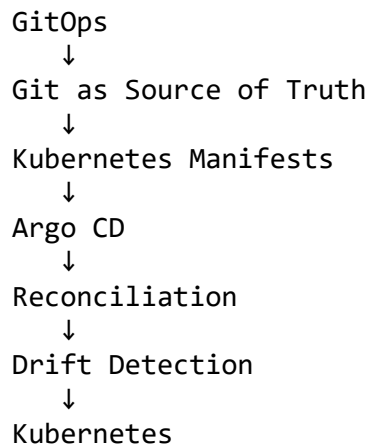


But this introduces another problem:

Who controls the cluster, and what happens if someone changes the cluster manually?

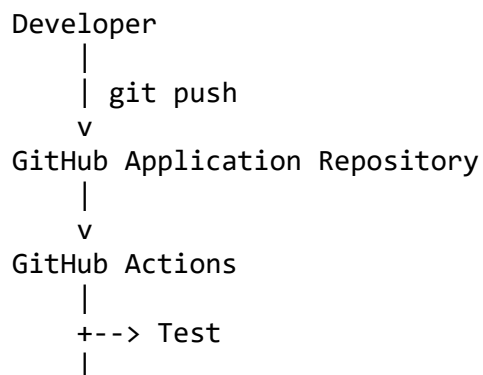
This leads us to **GitOps**.

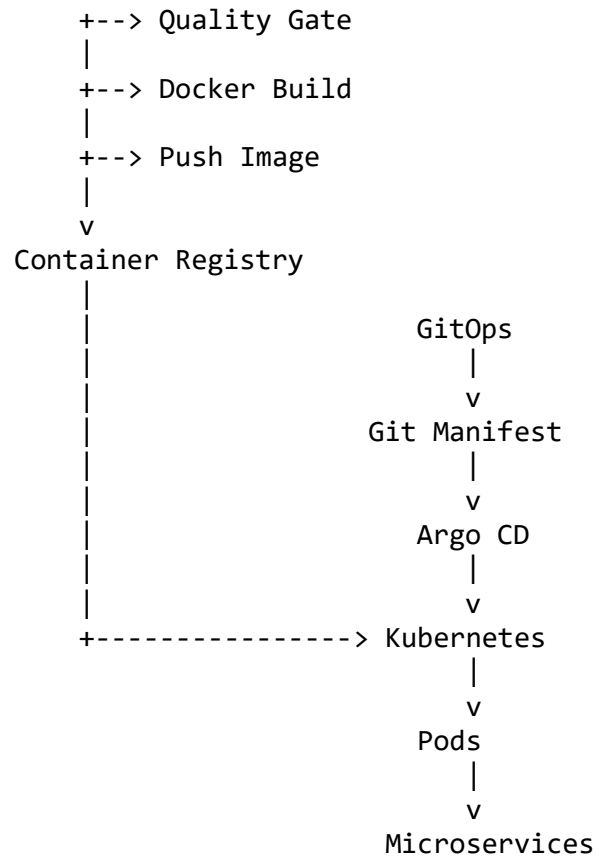
The next session will introduce:



The complete architecture will therefore become:

CI / CD ARCHITECTURE





Session 13 ends at the Container Registry.

Session 14 starts with GitOps and explains how the registry, Git manifests, Argo CD, and Kubernetes work together.